

FRED TALKS

30th June 2026

CONTENTS

Agent identity: a new access model for autonomous, team-wide AI
CLAUDE.COM · 1367 WORDS

How agents are transforming work | OpenAI
OPENAI · 1582 WORDS

Scaling Managed Agents: Decoupling the brain from the hands
ANTHROPIC ENGINEERING BLOG · 2024 WORDS

Giving LLMs a personality is just good engineering
SEANGOEDECKE.COM RSS FEED · 1062 WORDS

Voice In, Visuals Out @ AI Engineering World's Fair
ALLEN PIKE · 41 WORDS

Agent identity: a new access model for autonomous, team-wide AI

CLAUDE.COM · 24 JUN 2026 · [SOURCE](#)

For an AI agent to do its best work on a human-agent team, it needs access to the same tools, documents, and context humans have.

In a “single player” AI experience (where one person chats with one assistant), that’s straightforward: you connect your own accounts and the agent acts on your behalf. But in a “multiplayer” AI experience like [Claude Tag](#), Claude sits in a shared channel alongside many people at once, and it draws on the tools and context that belong to the *workspace*, rather than any one individual.

To make multiplayer experiences work, Claude needs its own accounts for those tools, set up by an admin and tied to the workspace. We call this access model **agent identity**.

In this post, we explain how agent identity works, how it moves permissions from per-user to per-channel, and how to scope it well in your own workspace.

Why “act as the user” breaks down

When you use AI as a personal assistant, you can connect platforms like Google Drive, GitHub, and your calendar, and let the model use *your* access permissions to read and write in them.

This model doesn’t work for Claude Tag for two reasons:

- **Increasing agent autonomy.** The length of a task that an AI agent can reliably complete on its own has been doubling roughly every four months. Agents now schedule their own tasks for later and respond to events long after the person who asked has logged off. While users set up routines that trigger them to act given certain situations, the agent works largely autonomously.
- **Multiplayer teams.** Claude Tag places Claude in shared spaces where teams are already working—e.g., a channel where three engineers and a PM are debugging together. But when more than one person is steering, whose permissions apply? There’s no single choice of

person that'd be right all of the time. This gives admins the ability to define what an agent can do in Slack independent from the humans involved, and a distinct tracking of what is done in Slack.

Claude acts as itself

In a channel where Claude Tag is active, Claude isn't acting on behalf of a single user. It has its own account in each system it touches: it posts in Slack as the Claude app, opens pull requests as the Claude GitHub App, and queries your warehouse under a service account provisioned by an admin.

And because there are no personal user credentials in play, a shared channel can never become a side door into someone's private documents.

Inheriting permissions

In the agent identity model, admins define an identity—the baseline set of connections and skills Claude holds everywhere—at the workspace level, and every channel inherits it by default. Then, where it makes sense, they can override it at the channel level, such as by granting the engineering channel access to GitHub and the data warehouse, or confining a CRM connection to a single private channel.

In addition to credentials, admins also define:

- **Repository access:** which repos Claude can read and write to.
- **Connectors:** the tools and API keys that Claude uses to do its job. Across an organization, different API keys can connect to the same service at different permission levels (e.g., Claude might be given read-only warehouse access in a general channel, and write access in the data team's private one).
- **Skills and plugins:** folders of instructions, scripts, and resources Claude loads dynamically to improve performance on specialized tasks.
- **Standing instructions:** custom instructions and context for each channel.

Because this model works around distinct Claude identities, revoking the identity ends Claude's access everywhere that the identity was used. This takes much less effort to manage than auditing individual agent actions across dozens of user accounts.

How the agent identity model works

Agent identity replaces the question “what can this user do?” with “what can this agent do in this compartment?” That’s a departure from per-user Access Control Lists: it means that a channel member without direct access to the repo can ask Claude to read that repo, if the channel’s profile grants Claude that permission.

This is unusual, but we think it is a necessary step toward an access model that works for autonomous, multiplayer agents. Below, we sketch out how to think about setting those boundaries.

How identity boundaries work

Claude Tag creates a distinct identity for each private channel; public channels in a workspace share a workspace-level identity. Claude's identity in a legal channel can't reach code that wasn't granted there, and its identity in an engineering channel can't read legal documents that weren't granted there. Memory and access respect those boundaries: what Claude learns in a private channel never appears in the wider workspace.

The identity belongs to the channel, so anyone in it can tag Claude by default, and admins can scope each channel's profile to the least-privileged member. On Enterprise plans, role-based access control lets admins go further and decide which members can invoke Claude at all, so a channel governs both what the agent can reach and who can ask.

Broad default access to tools and context

How teams scope Claude's tool access in Claude Tag:
broad, low-risk integrations run in shared channels under
an agent identity, while personal or team-specific tools stay
in DMs and run as the user.

Running Claude Tag inside Anthropic, we found that its value compounds with tool and context access. Each connected system makes every other one more useful, because Claude can combine context across them—pulling a thread from Slack, a doc from Drive, a ticket from a tracker, and a query from a warehouse into one answer that no single tool could provide.

The teams that get the most out of Claude are the ones that grant it generous access from the start, and pare access back depending on their organization’s admin preferences. Agent identity gives admins broad enough scope for Claude to do useful cross-system work, with boundaries

firm enough that the access never travels somewhere it wasn't granted. Our advice is to start with a baseline profile in a few channels, read the audit trail, and then extend access where the work justifies it, one deliberate grant at a time.

For organizations that require even more granularity, admins can disable Claude Tag in specific channels. Admins can also apply role-based access controls (RBAC) to limit access to Claude Tag to specific users.

Direct messages

With Claude Tag, direct messages work differently than in shared channels. DMs run on users' individual claude.ai accounts—their connectors, credentials, and name on the result. This makes DMs the right place to work with Claude on tasks and with tools that should never live in a channel, like email drafts or software only you have a license for.

Security and audit

When an admin adds a connection to a channel's profile, the credential is stored independently and mapped to that channel's identity, then injected at the network boundary at request time. Outbound traffic to any host an admin hasn't allowed is blocked outright. On the audit side, every routine, memory write, and network call made with agent credentials is recorded, and because Claude acts under its own service accounts, those actions also land in each connected system's own logs.

What's next

Agent identity is the foundation of Claude Tag's access model. In the future, we plan to strengthen our Claude Tag's security offerings to include just-in-time credential grants—so that a user can approve a single sensitive action in the moment without permanently widening the agent's scope—and an identity-aware overlay for organizations with more complex clearance structures. This will add user-level checks on top of an agent's scope, so Claude only acts when both the channel's profile and the requesting user's own permissions allow it.

The shift from single player to multiplayer AI in products like Claude Tag makes long-running, team-based work possible. Agent identity ensures that Claude's access to tools is broad enough to be useful, but scoped enough to be secure at enterprise scale.

Learn more about Claude Tag.

This article was written by Noah Zweben, a member of technical staff on the Claude Code team.

How agents are transforming work | OpenAI

OPENAI · 28 JUN 2026 · [SOURCE](#)

June 25, 2026

[Company](#)

A new Economic Research paper measuring Codex's economic potential at the frontier.

[Read the paper](#)

Listen to article

[AUDIO OMITTED]

Share

Agentic AI changes the unit of knowledge work from single interactions to delegated, long-horizon tasks. Chatbot interactions are often short and self-contained. Agents can operate independently for minutes or hours while orchestrating tool calls, interacting with environments, and iterating towards solutions. As a result, agents are quickly becoming the most powerful AI tool for work.

Over the last year, we witnessed this transformation first-hand at OpenAI. For the first few months after Codex was released to the public, ChatGPT remained the default AI tool for work within OpenAI. Through August 2025, the average OpenAI worker spent less than 10% of their tokens on Codex. Now, every department, including non-technical departments such as Legal and Recruiting, uses Codex as their primary AI tool for work. This pattern reflects what we believe will be the future of work given the expanded capabilities and accessibility of agentic tools.

Some content could not be imported from the original document. [View content ↗](#)

Some content could not be imported from the original document. [View content ↗](#)

Codex adoption grew in tandem with Codex’s capabilities. As Codex leveraged stronger models and new product features, it became capable of taking on an expanding set of productive tasks. Across Individual users, Organizational users, and OpenAI workers, we document four trends over the past year:

- **People use Codex for longer-horizon work.** By May 2026, 80.6% of sampled individual users made at least one Codex request estimated to exceed 30 minutes of human work, 70.2% made one estimated to exceed one hour, and 25.6% made at least one Codex request estimated to exceed eight hours.
- **Codex became the primary AI tool for every department at OpenAI.** Engineering moved first, but Legal, Finance, and Recruiting crossed into Codex being their primary AI tool around April 2026. For the average OpenAI worker, Codex usage now accounts for more than 85% of output tokens. Since Codex users tend to use more tokens than non-users, its share of overall tokens is even higher: Codex accounts for 99.8% of weekly output tokens generated within OpenAI.
- **Non-developer adoption grew especially rapidly, outpacing developer adoption.** Since August 2025, non-developer users rose 137x for individual users, 189x for organizational users, and 12x within OpenAI.
- **Codex enabled OpenAI workers to do tasks outside their job description.** While technical usage is still most prevalent among engineers, non-technical users regularly use Codex to take on coding or technical execution, including automation, data transformation, tooling, debugging, and structured analysis.

Agents work longer hours on harder tasks

Nearly a quarter of all Codex requests are for tasks that would take a person more than one hour to complete⁽¹⁾. As Codex’s capacity for independent long-context work improved, users shifted from short interactions toward more difficult tasks with longer horizons.

The chart below estimates the share of individual users that crossed four human-time thresholds: tasks that would take a person more than 30 minutes, more than one hour, more than four hours, and more than eight hours⁽²⁾. From December 2025 to May 2026, the share of users who made a request that was estimated to correspond to work that would take a person more than 30 minutes rose to 80.6%. The share making a request that would take a person more than one hour rose to 70.2%. The share requesting work that would take a person more than eight hours grew the fastest from a low base.

The growth in agentic usage can be seen in daily Codex runtime. Among daily active users at OpenAI, the heaviest users ask Codex to run many hours of agent work in a single day. By June 2026, users at the 99th percentile regularly generated more than 60 hours of Codex agent turns per day, distributed across multiple, parallel agents. As Codex became more powerful and parallelizable, users moved from only asking Codex for one answer at a time into increasingly orchestrating multiple agent tasks over the course of a day.

Adoption continues to move from engineers to the rest of OpenAI

Engineers at OpenAI began adopting Codex first, gradually. The average engineer at the company shifted the majority of their usage of OpenAI products to Codex by December 2025. Today, the average engineer generates 99% of their output tokens with Codex rather than ChatGPT. Legal, finance, and recruiting crossed over to majority use of Codex later, around April 2026, but their transitions were much faster. The average lawyer or recruiter at OpenAI now generates more than 85% of their output tokens on Codex.

Over the last six months, Codex usage has deepened and intensified at OpenAI. Among active internal users, change in combined output tokens rose sharply across departments. Research saw the biggest jump: by June 2026, median use was 56 times higher than in November 2025. Customer Support rose 32 times and Engineering rose 27 times, while Legal grew more gradually but still reached 13 times its November level.

These two patterns together illustrate how Codex has transformed how OpenAI uses AI to do productive work. Across the company, users are switching from chatbots to agents as their primary form of AI interaction and are deploying an exponentially growing amount of agentic labor.

Non-developers are the fastest-growing user groups

Across all user groups—OpenAI, organizational, and individual users—Codex use began with developers, the natural target audience for what began as a coding tool. However, as Codex expanded toward more general knowledge work, adoption among non-developers grew even more quickly. As shown in the user growth chart below, weekly non-developer users rose faster than developer users among individual, organizational, and OpenAI populations. By early June 2026, non-developer individual users multiplied 137 times since August 2025. Non-developer organizational users increased 189-fold, and non-developer OpenAI users increased 12-fold, likely because this group already started at a well above average starting point.

The shift does not mean every non-developer is using Codex in the same way as an engineer. Rather, it means that more non-developers are using Codex for some kind of agentic work.

Codex is expanding the horizon of potential productive work

Codex enables non-technical departments to accelerate workflows previously bottlenecked by technical expertise. The heat map below compares inferred occupations within OpenAI to the type of work represented in Codex outputs. Engineering and coding show up as the largest category for data science and research, whereas knowledge work is the largest category for finance and business operations, marketing, operations, and other departments.

That said, agentic tools can expand what an individual worker can do. For instance, over one-fourth of work done with Codex by workers in business functions was engineering or coding. Agents can lower the cost of moving across task boundaries and help workers do adjacent work that used to require more specialized technical support.

Occupation vs. work done with Codex

	Work category				
Consolidated inferred department	Engineering / Data		Financial Knowledge		Other
	Coding	Analysis	Analysis	Work	
Engineering	72%	4%	1%	18%	5%
Data					
Science /	51%	10%	0%	30%	9%
Research					
Finance / Biz	31%	9%	16%	34%	10%
Ops					
Product /					
Marketing /	25%	3%	7%	51%	15%
Ops					
Other	50%	7%	2%	38%	4%

Share of output tokens within department

What this means for agents' economic potential

Increased use of agentic tools by non-engineer employees expands the frontier of what these workers can do. That matters for businesses deciding how to redesign workflows, for employees learning which skills become more valuable, and for policymakers and researchers trying to understand how AI changes the labor market.

Our paper shows how frontier users adopt capable agentic tools at the frontier. Our results demonstrate what unfolds when people have broad, low-friction access to capable agentic tools: as the tools improve, people use them for longer, more complex, and more cross-functional work. As time goes on, this is likely to be what the future of work looks like.

- [2026](#)
- [Economic Research](#)
- [Codex](#)

Author

OpenAI

Footnotes

1. Task horizon is estimated using an LLM-as-judge with access to Codex transcripts.

[SVG OMITTED]

[\[find in text\]](#)
2. These thresholds are model-estimated, so they should be treated as directional rather than exact. This is also based only on individual data and based on the queries of a random sample of 0.1% of users.

[SVG OMITTED]

[\[find in text\]](#)

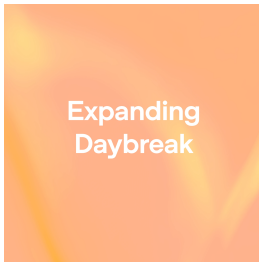
Keep reading

[View all](#)



OpenAI and Broadcom unveil LLM-optimized inference chip

CompanyJun 24, 2026



Daybreak: Tools for securing every organization in the world

SecurityJun 22, 2026



Samsung Electronics brings ChatGPT and Codex to employees

CompanyJun 21, 2026

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 24 APR 2026 · [SOURCE](#)

Get started with Claude Managed Agents by following our [docs](#).

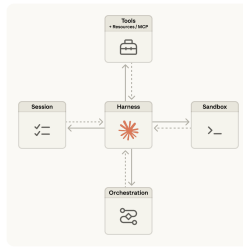
A running topic on the Engineering Blog is how to [build effective agents](#) and [design harnesses](#) for [long-running work](#). A common thread across this work is that harnesses encode assumptions about what Claude can't do on its own. However, those assumptions need to be frequently questioned because they can [go stale](#) as models improve.

As just one example, in prior work [we found](#) that Claude Sonnet 4.5 would wrap up tasks prematurely as it sensed its context limit approaching—a behavior sometimes called “context anxiety.” We addressed this by adding context resets to the harness. But when we used the same harness on Claude Opus 4.5, we found that the behavior was gone. The resets had become dead weight.

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.

Building Managed Agents meant solving an old problem in computing: how to design a system for “[programs as yet unthought of](#).” Decades ago, operating systems solved this problem by virtualizing hardware into abstractions—*process*, *file*—general enough for programs that didn't exist yet. The abstractions outlasted the hardware. The `read()` command is agnostic as to whether it's accessing a disk pack from the 1970s or a modern SSD. The abstractions on top stayed stable while the implementations underneath changed freely.

Managed Agents follow the same pattern. We virtualized the components of an agent: a session (the append-only log of everything that happened), a harness (the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure), and a sandbox (an execution environment where Claude can run code and edit files). This allows the implementation of each to be swapped without disturbing the others. We're opinionated about the shape of these interfaces, not about what runs behind them.



We started by placing all agent components into a single container, which meant the session, agent harness, and sandbox all shared an environment. There were benefits to this approach, including that file edits are direct syscalls, and there were no service boundaries to design.

But by coupling everything into one container, we ran into an old infrastructure problem: we'd adopted a *pet*. In the pets-vs-cattle analogy, a pet is a named, hand-tended individual you can't afford to lose, while cattle are interchangeable. In our case, the server became that pet; if a container failed, the session was lost. If a container was unresponsive, we had to nurse it back to health.

Nursing containers meant debugging unresponsive stuck sessions. Our only window in was the WebSocket event stream, but that couldn't tell us *where* failures arose, which meant that a bug in the harness, a packet drop in the event stream, or a container going offline all presented the same. To figure out what went wrong, an engineer had to open a shell inside the container, but because that container often also held user data, that approach essentially meant we lacked the ability to debug.

A second issue was that the harness assumed that whatever Claude worked on lived in the container with it. When customers asked us to connect Claude to their virtual private cloud, they had to either peer their network with ours, or run our harness in their own environment. An assumption baked into the harness became a problem when we wanted to connect it to different infrastructure.

The solution we arrived at was to decouple what we thought of as the "brain" (Claude and its harness) from both the "hands" (sandboxes and tools that perform actions) and the "session" (the log of session events). Each became an interface that made few assumptions about the others, and each could fail or be replaced independently.

The harness leaves the container. Decoupling the brain from the hands meant the harness no longer lived inside the container. It called the container the way it called any other tool: `execute(name, input) → string`. The container became cattle. If the container died,

the harness caught the failure as a tool-call error and passed it back to Claude. If Claude decided to retry, a new container could be reinitialized with a standard recipe: `provision({resources})`. We no longer had to nurse failed containers back to health.

Recovering from harness failure. The harness also became cattle. Because the session log sits outside the harness, nothing in the harness needs to survive a crash. When one fails, a new one can be rebooted with `wake(sessionId)`, use `getSession(id)` to get back the event log, and resume from the last event. During the agent loop, the harness writes to the session with `emitEvent(id, event)` in order to keep a durable record of events.

Component	Interface (methods)	Satisfied by
Session	<code>perform(session_id, op)</code> → <code>Stream<Event></code> <code>perform(session_id, op)</code> → <code>ResultingEvent[]</code> (if not yet committed) <code>setEvent(session_id, event)</code>	Any agent only log that can be committed or not, any event passed and accepted, interrupt, append, ... Plugins, K8s, in-memory, etc.
Orchestration	<code>watch(session_id) → void</code>	Any scheduler that can call a function with op to watch on session - a cron job is a good example, or using lang 40.
Harness	<code>watchEffect() → EffectResult[]</code>	Any logic that can affect and depends on events in the session.
Sandbox	<code>provision({resources})</code> <code>cancel(session_id)</code> → <code>ResultingEvent[]</code> <code>provision({resources})</code> → <code>ResultingEvent[]</code>	Any executor that can be verified and executed after any event is triggered, a good example is using container, etc.
Resource	<code>claim(session_id, name, part)</code>	Any executor that can be verified and executed after any event is triggered, a good example is using container, etc.
Tools	<code>claim(session_id, name, part)</code>	Any executor that can be verified and executed after any event is triggered, a good example is using container, etc.

The security boundary. In the coupled design, any untrusted code that Claude generated was run in the same container as credentials—so a prompt injection only had to convince Claude to read its own environment. Once an attacker has those tokens, they can spawn fresh, unrestricted sessions and delegate work to them. Narrow scoping is an obvious mitigation, but this encodes an assumption about what Claude can't do with a limited token—and Claude is getting increasingly smart. The structural fix was to make sure the tokens are never reachable from the sandbox where Claude's generated code runs.

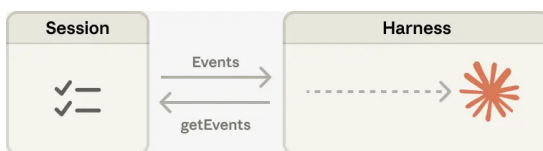
We used two patterns to ensure this. Auth can be bundled with a resource or held in a vault outside the sandbox. For Git, we use each repository's access token to clone the repo during sandbox initialization and wire it into the local git remote. Git `push` and `pull` work from inside the sandbox without the agent ever handling the token itself. For custom tools, we support MCP and store OAuth tokens in a secure vault. Claude calls MCP tools via a dedicated proxy; this proxy takes in a token associated with the session. The proxy can then fetch the corresponding credentials from the vault and make the call to the external service. The harness is never made aware of any credentials.

The session is not Claude's context window

Long-horizon tasks often exceed the length of Claude's context window, and the standard ways to address this all involve irreversible decisions about what to keep. We've explored these techniques in [prior work](#) on context engineering. For example, compaction lets Claude save a

summary of its context window and the memory tool lets Claude write context to files, enabling learning across sessions. This can be paired with context trimming, which selectively removes tokens such as old tool results or thinking blocks.

But irreversible decisions to selectively retain or discard context can lead to failures. It is difficult to know which tokens the future turns will need. If messages are transformed by a compaction step, the harness removes compacted messages from Claude's context window, and these are recoverable only if they are stored. Prior work has explored ways to address this by storing context as an object that lives *outside* the context window. For example, context can be an object in a REPL that the LLM programmatically accesses by writing code to filter or slice it.



In Managed Agents, the session provides this same benefit, serving as a context object that lives outside Claude's context window. But rather than be stored within the sandbox or REPL, context is durably stored in the session log. The interface, `getEvents()`, allows the brain to interrogate context by selecting positional slices of the event stream. The interface can be used flexibly, allowing the brain to pick up from wherever it last stopped reading, rewinding a few events before a specific moment to see the lead up, or rereading context before a specific action.

Any fetched events can also be transformed in the harness before being passed to Claude's context window. These transformations can be whatever the harness encodes, including context organization to achieve a high prompt cache hit rate and context engineering. We separated the concerns of recoverable context storage in the session and arbitrary context management in the harness because we can't predict what specific context engineering will be required in future models. The interfaces push that context management into the harness, and only guarantee that the session is durable and available for interrogation.

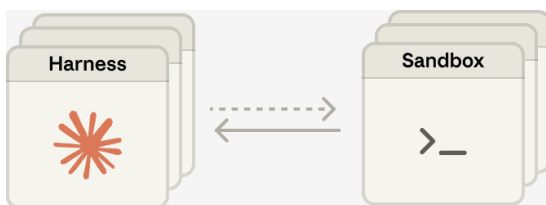
Many brains. Decoupling the brain from the hands solved one of our earliest customer complaints. When teams wanted Claude to work against resources in their own VPC, the only path was to peer their network with ours, because the container holding the harness assumed every resource sat next to it. Once the harness was no longer in the container, that assumption went away. The same change had a performance payoff. When we initially put the brain in a container, it meant that many brains required as many containers. For each brain, no inference could happen until that container was provisioned; every session paid the full container setup cost up front. Every session, even ones that would never touch the sandbox, had to clone the repo, boot the process, fetch pending events from our servers.

That dead time is expressed in time-to-first-token (TTFT), which measures how long a session waits between accepting work and producing its first response token. TTFT is the latency the user most acutely *feels*.

Decoupling the brain from the hands means that containers are provisioned by the brain via a tool call (`execute(name, input) → string`) only if they are needed. So a session that didn't need a container right away didn't wait for one. Inference could start as soon as the orchestration layer pulled pending events from the session log. Using this architecture, our p50 TTFT dropped roughly 60% and p95 dropped over 90%. Scaling to many brains just meant starting many stateless harnesses, and connecting them to hands only if needed.

Many hands. We also wanted the ability to connect each brain to many hands. In practice, this means Claude must reason about many execution environments and decide where to send work—a harder cognitive task than operating in a single shell. We started with the brain in a single container because earlier models weren't capable of this. As intelligence scaled, the single container became the limitation instead: when that container failed, we lost state for every hand that the brain was reaching into.

Decoupling the brain from the hands makes each hand a tool, `execute(name, input) → string`: a name and input go in, and a string is returned. That interface supports any custom tool, any MCP server, and our own tools. The harness doesn't know whether the sandbox is a container, a phone, or a Pokémon emulator. And because no hand is coupled to any brain, brains can pass hands to one another.



The challenge we faced is an old one: how to design a system for “programs as yet unthought of.” Operating systems have lasted decades by virtualizing the hardware into abstractions general enough for programs that didn't exist yet. With Managed Agents, we aimed to design a system that accommodates future harnesses, sandboxes, or other components around Claude.

Managed Agents is a meta-harness in the same spirit, unopinionated about the *specific* harness that Claude will need in the future. Rather, it is a system with general interfaces that allow many different harnesses. For example, Claude Code is an excellent harness that we use widely across tasks. We've also shown that task-specific agent harnesses excel in narrow domains. Managed Agents can accommodate any of these, matching Claude's intelligence over time.

Meta-harness design means being opinionated about the interfaces around Claude: we expect that Claude will need the ability to manipulate state (the session) and perform computation (the sandbox). We also expect that Claude will require the ability to scale to many brains and many hands. We designed the interfaces so that these can be run reliably and securely over long time horizons. But we make no assumptions about the number or location of brains or hands that Claude will need.

Written by Lance Martin, Gabe Cemaj and Michael Cohen. Special thanks to the Agents API team and Jake Eaton for their contributions.

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · [SOURCE](#)

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this [opinion piece](#) by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into [AI psychosis](#). Here's a representative paragraph from the piece:

In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system.**

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training¹.

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess². AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there³.

When AI people talk about LLMs having personalities, or wanting things, or even having souls⁴, these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained

on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a [recent paper](#) about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

↩

2. You could explain this in terms of “the stories we tell ourselves”. Many people (though [not all](#)) think that human identities are narratively constructed.

↩

3. I wrote about this last year in [Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality](#). A little nudge to change Grok's views on South African internal politics can cause it to start calling itself “Mecha-Hitler”.

↩

4. I have long believed that Claude “feels better” to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its “soul”). My guess is that if you tried to make a “less human” version of Claude, it would become rapidly less capable.

↩

Voice In, Visuals Out @ AI Engineering World's Fair

ALLEN PIKE · 29 JUN 2026 · [SOURCE](#)

This week's AI Engineering World's Fair just posted [my talk on the agony and ecstasy of voice in, visuals out agents](#). It's a challenge to get model responses that feel immediate, but when it works, it feels magical.